ENCE260: Embedded Systems

Lecture 9: Modularity

How to organise our code.

Revision

If `game.c` is modified, what does `make program` run?

```
# Definitions.
CC = avr-gcc
CFLAGS = -mmcu=atmega32u2 -Os -Wall -Wstrict-prototypes -Wextra -g -I. -I../../utils -I../../fonts -I../../drivers
    -I../../drivers/avr
OBJCOPY = avr-objcopy
SIZE = avr-size
DEL = rm

# Default target.
all: game.out

# Compile: create object files from C source files.
game.o: game.c ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

system.o: ../../drivers/avr/system.c ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

# Link: create ELF output file from object files.
game.out: game.o system.o
    $(CC) $(CFLAGS) $^ -o $@ -lm
    $(SIZE) $@

# Target: clean project.
.PHONY: clean
clean:
    -$(DEL) *.o *.out *.hex

# Target: program project.
.PHONY: program
program: game.out
    $(OBJCOPY) -O ihex game.out game.hex
    dfu-programmer atmega32u2 erase; dfu-programmer atmega32u2 flash game.hex; dfu-programmer atmega32u2 start
```

Revision

- `game.c` is modified and the target `program` is built.
- `program` requires `game.out`, which requires `game.o`, which requires `game.c`.
- The commands run are therefore:

```
avr-gcc -c -mmcu=atmega32u2 ... game.c -o game.o
avr-gcc -mmcu=atmega32u2 ... game.o system.o -o game.out -lm
avr-size game.out
avr-objcopy -O ihex game.out game.hex
dfu-programmer atmega32u2 erase;
dfu-programmer atmega32u2 flash game.hex;
dfu-programmer atmega32u2 start
```

Module Motivation

- A common way to group modules in C is to base them around a data type.
- You *should* remember this from last term!
- For example,

```c
#ifndef RECT_H
#define RECT_H
#include <stdint.h>

typedef struct {
    uint32_t width;
    uint32_t height;
} Rect;

void rect_set(Rect* rect, uint32_t width, uint32_t height);
uint32_t rect_area(const Rect* rect);
uint32_t rect_perimeter(const Rect* rect);

#endif //RECT_H
```

Module Guidelines

- Modules don't need to be built around a type!
- However, their functions **do** need to be logically related at the very least.
- Doing so is useful because it ensures that:
  - Modules make it easier to understand and maintain the code.
  - Modules can be re-used by other programs.
  - Modular programs are faster to compile than non-modular ones.
  - Modules can be tested independently of each other.
  - Your assignment marking schedule requires it ☺.

Abstraction

- *Abstraction* is the act of hiding details and specifics from the programmer using your module. We do this by separating the:

  Interface: The signatures of exposed functions, as well as any types, from the

  Implementation: Specific details (such as the body of the functions).

- In C, the interface is usually in the header file, and the implementation is usually in the `.c` file.
- For example, `led_on()` is a function that turns the LED on without the user needing to set the `DDRD` and `PORT` registers. The interface could be kept the same for many changes to the implementation.
- If we change microcontrollers, this might let us keep the same `main()` and only change `led.c` for a blinky program.

Encapsulation

- The grouping of related functions and data structures in a module is known as *encapsulation*.
- Encapsulation makes it easier to re-use modules for other programs.
- For example, if we want to re-use our display functions in another program, we put all the macros, type definitions and functions relating to the display in one file.
- If we want to use the display in a new program, then re-using a module with good encapsulation means fewer other files are needed.

Bad Example: Lab 1 Exercise 1

```c
#include <avr/io.h>
#include "system.h"

int main (void)
{
    system_init ();

    DDRC |= (1 << 2); // Magic Numbers!

    DDRD &= ~(1 << 7); // Hardware-specific implementation details!

    while (1) {

        // These checks and sets should be functions from modules!

        if ((PIND & (1<<7))!=0) {
            PORTC |= (1<<2);
        } else {
            PORTC &= ~(1<<2);
        }
    }
}
```

Good Example: Lab 1 Exercise 3

```c
#include "button.h"
#include "led.h"
#include "system.h"

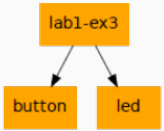

int main (void)
{
    system_init ();

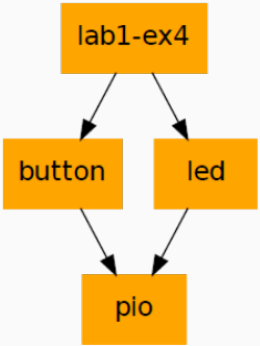
    led_init(); // Don't need to know which architecture.
    button_init(); // Don't need to know details.

    while (1)
    {
        if (button_pressed_p()) {
            led_on(); // Can change hardware and this file still works.
        } else {
            led_off();
        }
    }
}
```

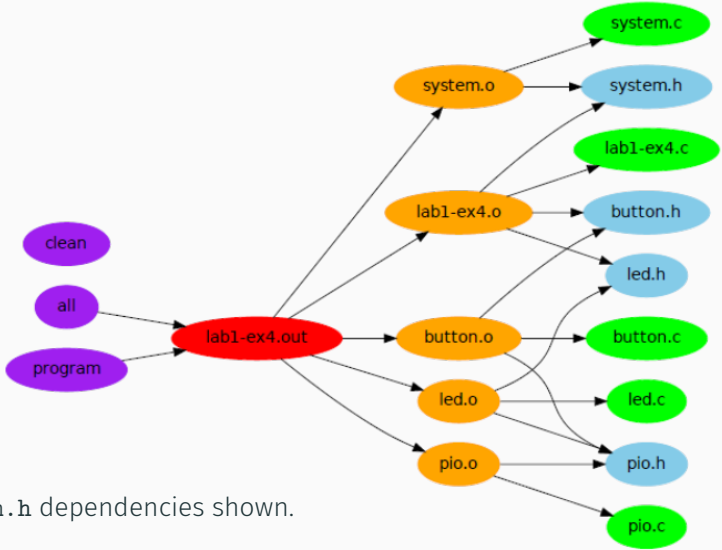
Abstraction Layers

- Additional modules can be used to reduce complexity.
- E.g., in Lab 1 exercise 4, we rewrote the button and LED modules to use the PIO module.
- This abstraction reduces the hardware dependence.
- If we use a different microcontroller, we just need to replace the `pio` module. The files from `button`, `led`, and `lab1-ex4` don't need to change.
- In practice we have many more layers, each of which is more complex than in this example.





Types of Dependency



Not all `system.h` dependencies shown.

Coupling

- Coupling refers to the degree of interconnectedness between different modules.
- If the modules are lowly (or loosely) coupled, a change in one module will have little or no effect on other modules. This is good.
- If the modules have high coupling, changes in one module will require changes to many other modules as well. This is bad. (e.g. globals; data should flow through functions).
- Low coupling aids the implementation and maintenance of the program, as different people can work on separate modules without negatively influencing each other.

Cohesion

- Cohesion is how closely related the components of a module are to each other.
- If each function, type, etc. in the module are closely related, and cohesion is high. If they are distinct, the cohesion is low.
- A module that only has functions to calculate `sin()`, `cos()`, and `tan()` has high cohesion. This is good, because it helps programmers navigate the code, and make encapsulation easier.
- Modules like an `init.h` have very low cohesion because they relate to almost every subsystem on the board. Keep e.g. everything related to your transmission logic in it's own module, including its `init()` function.

Public vs. Private

- Because the compiler and the linker are actually different programs, the only things a `.c` file sees are its included header files. `.c` file contents are hidden.
- This means only functions/types in header files are “exposed” or “public”.
- Anything that’s only in a single `.c` file is “private”. This can be enforced with the `static` keyword.
- By limiting how much we make public, we can keep our coupling low, which makes things easier for users of our modules.
- Keep as much out of the `.h` file as possible! This way it acts as clear documentation.

Top-Down Program Design

- First, we decompose the problem (e.g. game) into sub-problems (e.g. display, buttons, communication, ...).
- Then, do the same for all the sub-problems (e.g. display →setting pixels, remembering pixels, timing multiplexing, ...).
- Repeat this process until it's easier to write the code instead of splitting up the problem.

Pros: Code is easy to follow from most to least abstract, and you end up with well-defined functions.

Cons: Not all problems decompose neatly, and we can't test anything until the low-level modules are complete.

Bottom-Up Program Design

- It can be hard to get started with top-down design.
- In bottom-up design, we specify/develop low-level modules first, then combine them to achieve functionality.

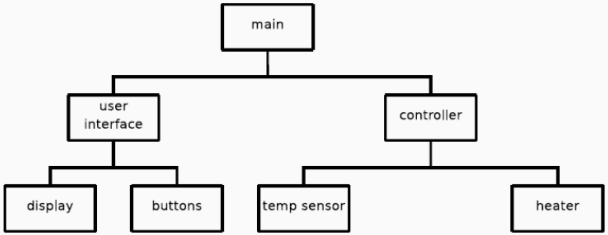
Pros: Reliably develops reusable modules, and those modules can be tested early.

Cons: You might end up with the wrong building blocks, so you may not be able to meet the overall goal.

- In practice, we combine both top-down and bottom-up methods into a hybrid approach.
- The real world also requires much more iteration.

Program Design Example

- Say you're developing the firmware for a rice cooker.
- Regardless of method, a good program structure should look something like this:



- If our rice cooker overcooks the rice, we can iterate by modifying only the controller.
- Good program design will let us do this without changing other modules.

Program Design Checklist

Questions to ask yourself about your program:

- Does each module have a single purpose? (Encapsulation)
- Does the module hide implementation details from other modules? (Abstraction)
- Does each module contain a strongly related set of functions? (High Cohesion)
- Is each module maximally independent from other modules? (Low Coupling)

Key Concepts

- Code should be written in modules that logically clump together, and interact as little as possible with other modules.
- This minimizes programmer overload, maximizes reusability, and makes it easier to develop programs.
- Good program design results in limited dependencies that only flow one way.

Homework

You *should* have started your assignment by now. Take some time to review your current code structure and improve it to follow these principles now.

If you've not yet started, do so by trying some explicit program design before you start coding.

It is significantly easier to structure your code well from the start.

Trying to make the code nice and modularised at the end will take much longer overall.